

PAGE 2026 — {ospsuite} + {esqlabsR} Cheatsheet

Workshop reference — 2 pages

Pavel Balazki · Wilbert de Witte

2026-06-01

{ospsuite} essentials

Workflow

load → inspect → modify → run → extract → analyze
→ plot

Load + save

```
sim <- loadSimulation("path/Model.pkml")
saveSimulation(sim, "path/Model_v2.pkml")
# Independent copy? Reload - sim$clone() is broken
# for ospsuite (.NET-backed R6).
sim_copy <- loadSimulation("path/Model.pkml")
```

Inspect

```
getAllParameterPathsIn(sim)
getAllMoleculePathsIn(sim)
getAllContainerPathsIn(sim)
getAllParametersMatching("**|Volume", sim)
```

Parameter access

```
p <- getParameter("Organism|Liver|Volume", sim)
p$value; p$unit
setParameterValuesByPath("Organism|Weight", 75, sim, units =
  ↪ "kg")
```

Run

```
res <- runSimulations(sim)[[1]] # single
res_list <- runSimulations(list(sim1, sim2)) # batch
```

Extract

```
out <- getOutputValues(res, quantitiesOrPaths =
  ↪ "...|Concentration")
out$data # → data.frame: Time, paths, values
```

Plot — DataCombined

```
dc <- DataCombined$new()
dc$addSimulationResults(res)
dc$addObservedData(loadDataSetFromPKML("obs.pkml"))
plotIndividualTimeProfile(dc) # time profile
plotObservedVsSimulated(dc) # GoF
plotResidualsVsTime(dc) # residuals
```

PK params

```
pk <- calculatePKAnalyses(res, "...|Concentration")
pkAnalysesAsDataFrame(pk) # AUC, C_max, t_max, t½
addUserDefinedPKParameter(...)
```

Local sensitivity

```
sa <- sensitivityCalculation(
  simulation = sim,
  variableParameterPaths = c("...|Lipophilicity",
    "...|GFR_filtration_fraction"),
  variationRange = 0.1)
mostSensitiveParameters(sa, totalSensitivityThreshold = 0.9)
```

Units

```
toUnit("Concentration", 1, "mg/L")
toBaseUnit("Volume", 70, "kg")
toDisplayUnit(p, p$value)
```

Path conventions

Aciclovir|Organism|Liver|Volume

Levels: organ → compartment → parameter. | separates levels. ** = recursive wildcard.

Parameter identification

```
library(ospsuite.parameteridentification)
```

Optimization parameter

```
piPar <- PIParameters$new(
  parameters = getParameter("Aciclovir|Lipophilicity", sim))
piPar$minValue <- -5
piPar$maxValue <- 5
piPar$startValue <- 2 # optional
```

Observed data → output mapping

```
obs <- loadDataSetsFromExcel(
  "Data/Obs.xlsx", projectConfiguration = project)

simPath <- "Organism|PeripheralVenousBlood|Aciclovir|Plasma"
  ↪ (Peripheral Venous Blood)
mapping <- PIOutputMapping$new(
  quantity = getQuantity(simPath, container = sim))
mapping$addObservedDataSets(obs$`Aciclovir.Vergin1995.IV250`)
```

Configure + run

```
cfg <- PIConfiguration$new()
cfg$algorithm <- "DEoptim" # or "LevenbergMarquardt"
cfg$autoEstimateCI <- TRUE
```

```
piTask <- ParameterIdentification$new(
  simulations = sim,
  parameters = piPar,
  outputMappings = mapping,
  configuration = cfg)
```

```
res <- piTask$run()
piTask$plotResults()
```

{esqlabsR} essentials

Project structure

```
MyProject/  
  ProjectConfiguration.xlsx      ← master  
  Models/  
    Simulations/      *.pkml  
  Configurations/  
    Scenarios.xlsx  
    ModelParameters.xlsx  
    Applications.xlsx  
    Plots.xlsx  
  Data/                *.xlsx  
  Results/  
  Reports/              *.qmd
```

Initialize

```
library(esqlabsR)  
initProject(path = "~/MyProject")
```

Project object

```
project <-  
  ↪ createProjectConfiguration("ProjectConfiguration.xlsx")  
project$paths$ModelsFolder  
project$paths$ConfigurationsFolder
```

Load + run scenarios

```
cfg <- readScenarioConfigurationFromExcel(  
  scenarioNames = c("P0_7.5mg", "IV_2mg"),  
  projectConfiguration = project)  
sc <- createScenarios(cfg)  
res <- runScenarios(scenarios = sc)
```

Snapshot / restore

```
snapshot(project)      # Excel → JSON snapshot  
restore(project)       # JSON → Excel  
status(project)        # diff
```

Plots from Excel

```
plots <- createPlotsFromExcel(  
  plotGridNames = c("P0_vs_obs"),  
  simulatedScenarios = res,  
  observedData = obs,  
  projectConfiguration = project)  
plots[[1]]
```

Sensitivity (multi-range)

```
simulation <- scenarios$P0_7.5mg$simulation  
analysis <- sensitivityCalculation(  
  simulation = simulation,  
  outputPaths = c(Plasma = "..."),  
  parameterPaths = c(Lipo = "Midazolam|Lipophilicity"),  
  variationRange = c(0.1, 0.5, 1, 2, 10),  
  pkParameters = c("C_max", "AUC_inf"))  
sensitivitySpiderPlot(analysis)  
sensitivityTornadoPlot(analysis)  
sensitivityTimeProfiles(analysis)  
saveSensitivityCalculation(analysis, outputDir = "Results/SA")
```

Excel cheat-row reference

File	One row =	Key columns
Scenarios	one scenario	Scenario, ModelFile, ApplicationProtocol, IndividualId, OutputPathsId
ModelParameters	parameter override	Id, ContainerPath, ParameterName, Value, Units
Applications	dosing protocol	Id, DrugMass, DoseUnit, StartTime, Route
Plots	one figure	PlotID, ScenarioName, OutputPath, xLog, yLog

Quarto report — re-render

```
quarto render Reports/MyReport.qmd      # default  
quarto render Reports/MyReport.qmd -P dose_mg:15  # override  
↪ param  
quarto render Reports/MyReport.qmd --to pdf      # PDF only
```

In a chunk

```
# parameters from YAML  
params$dose_mg  
  
# auto-run scenarios  
cfg <- readScenarioConfigurationFromExcel(params$scenario,  
  ↪ project)  
sc <- createScenarios(cfg)  
res <- runScenarios(sc)  
plots <- createPlotsFromExcel(plotGridNames = "MyPlot",  
  simulatedScenarios = res,  
  projectConfiguration = project)
```

Repo

r-training.esqlabs.com github.com/esqLABS/r-
packages-training